Smart Traffic Surveillance System Using Computer Vision, Ground Plane Homography, and Tableau Analytics

Naval C N*, Emmanuel Lara†, Kalpesh Kashyap‡, Prof. Gargi N§, Prof. Bhavani Soma¶
Department of Artificial Intelligence and Data Science, Global Academy of Technology
Bengaluru, Karnataka, India
Email: *1ga23ad034@gat.ac.in, †1ga23ad015@gat.ac.in, ‡1ga23ad022@gat.ac.in, §gargi.n@gat.ac.in, ¶bhavani.soma@gat.ac.in

***Abstract*—Traffic congestion and over-speeding are big problems in modern cities. Most traditional methods like radar guns and road loop sensors are expensive to install and fix. Standard roadside CCTV cameras are already available everywhere, but estimating vehicle speed from a normal camera video is tricky because the road is seen at an angle. In this view, objects far away look very small and move fewer pixels compared to objects closer to the camera. If we calculate speed by simply counting pixels, the calculated speed will be completely wrong. In this paper, we built an easy-to-use traffic monitoring pipeline. We use YOLOv8n to detect vehicles like cars and motorcycles in each frame. We then use ByteTrack to give each vehicle a unique tracking ID so it is not counted twice. To fix the camera angle problem, we apply a 4-point planar homography transform. This maps the bottom contact point of each vehicle from screen pixels directly to real-world meters on the road. The calculated speeds are smoothed using an exponential filter to remove box jumping. We also set a virtual counting line across the road to record vehicle flow over time. All the results are saved into CSV files and visualized in an interactive Tableau dashboard. The dashboard shows real-time metrics, speed distributions, class counts, and highlights over-speeding vehicles. In our test video, the system tracked 37 vehicle tracks, found an average fleet speed of 41.24 km/h, and flagged 5 over-speed instances above the 60 km/h speed limit.**

***Index Terms*—Traffic Monitoring, YOLOv8, ByteTrack, Planar Homography, Speed Estimation, Tableau Dashboard, Data Visualization.**

I. Introduction

Managing road traffic in growing cities needs continuous monitoring. Traffic police and city authorities need to know two main things every day: how many vehicles are using a road, and whether drivers are following the speed limits.

Physical sensors, like magnetic induction loops cut into the road surface or roadside Doppler radar units, work well but have clear downsides. They cost a lot of money to set up, require digging up the road, and break down under heavy weather or road repairs. On the other hand, traffic cameras are already mounted on poles and bridges across most highways. If we can process video feeds from these cameras directly using computer vision, we get a low-cost and flexible traffic monitoring tool.

However, writing software to do this on regular camera video has practical problems:

1) **Perspective effect:** Cameras look down at the road at a slant. A car moving 10 meters when far away only shifts a few pixels on screen, while the same car moving 10 meters near the bottom shifts hundreds of pixels. A simple pixel-to-meter multiplier gives false speeds (often over 200 km/h).
2) **Tracking through occlusions:** When traffic is close together, one car can briefly block another. The tracking algorithm must keep the same vehicle ID instead of creating duplicate counts.
3) **Usable reports:** Raw numbers and video boxes alone are not helpful for operators. We need clean summary charts, speed trends, and violation lists that can be read at a glance.

To solve these problems, we created a complete mini-project system. We combined YOLOv8n for fast object detection, ByteTrack for vehicle tracking, perspective homography for accurate speed calculation in meters, and Tableau for visual analytics. The whole system runs on ordinary computers without needing specialized hardware.

II. Related Work

Vehicle detection in computer vision has improved rapidly with deep learning. Earlier methods used hand-crafted features like Haar cascades or HOG combined with SVM. These were slow and often failed when shadows or lighting changed.

The introduction of convolutional neural networks changed this. Two-stage detectors like Faster R-CNN gave high accuracy but were too heavy for real-time video. Single-stage detectors such as the YOLO series (YOLOv3, YOLOv4, and YOLOv8) process full images in one pass, making them fast enough for real-time traffic monitoring.

For tracking moving objects from frame to frame, the tracking-by-detection method is commonly used. SORT used Kalman filters and bounding box overlap (IoU) to follow objects. ByteTrack improved on SORT by keeping low-confidence detection boxes instead of immediately throwing them away. This simple change allows it to track vehicles even when they are partly hidden behind another vehicle.

For measuring speed from a single camera, researchers use camera calibration. Standard two-line tripwire systems place

two virtual lines on the screen and divide the distance by the elapsed time. But if the camera is tilted, assuming equal distance between lines produces large errors. Homography and Inverse Perspective Mapping (IPM) use geometric transform matrices to map tilted road planes to flat top-down views. This gives a much more reliable metric measurement.

Finally, displaying traffic data in visual analytics tools like Tableau or PowerBI helps city planners inspect rush-hour patterns, lane usage, and violation hotspots through interactive charts.

III. Proposed Methodology

Our complete system consists of four clear steps: vehicle detection, multi-vehicle tracking, homography speed calculation, and Tableau visual reporting. Fig. 1 shows how data moves through the pipeline.

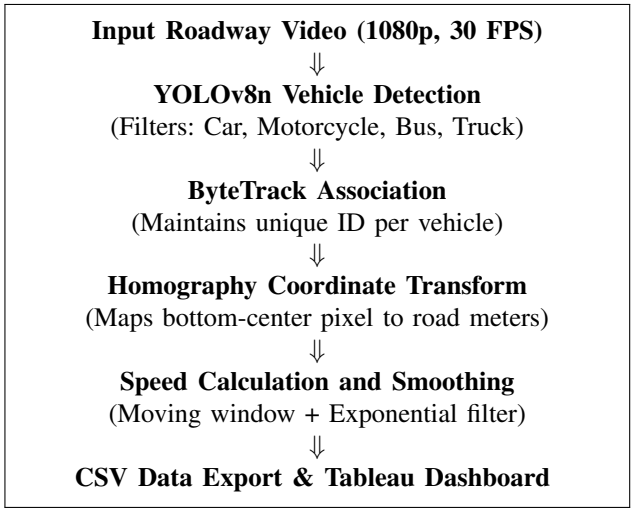


Fig. 1. Step-by-step block diagram of our traffic surveillance system.

A. Vehicle Detection Using YOLOv8n

We use the pre-trained YOLOv8 nano model (YOLOv8n) because it runs fast even without high-end GPUs. Each video frame has a resolution of 1920×1080 pixels at 30 frames per second.

We only keep detections that belong to common road transport classes:

$$\text{Classes} = \{\text{car}, \text{motorcycle}, \text{bus}, \text{truck}\} \tag{1}$$

We set a detection confidence threshold of 0.30. Bounding boxes below this threshold are ignored to avoid false alarms from road signs or roadside structures.

B. Object Tracking with ByteTrack

Detecting a vehicle in each frame independently is not enough. We need to follow the same car across continuous frames to measure its motion. We use ByteTrack as our tracker.

ByteTrack keeps a track record for each vehicle using a Kalman filter. When a new frame arrives:

1) It first matches high-confidence detections with existing tracks using bounding box overlap (IoU).
2) Then, it matches the remaining tracks with lower-confidence detections. This prevents a track from getting lost when a car is temporarily hidden by a tree branch or another vehicle.

Each vehicle keeps its assigned integer ID until it leaves the camera view.

C. Planar Homography Speed Estimation

In a tilted camera shot, pixel distances cannot be directly converted to meters using a single number. Instead, we use planar homography.

A homography matrix $\mathbf{H}$ of size 3×3 relates any point (u, v) on the image plane to a point (X, Y) on the flat road surface:

$$\begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} \sim \mathbf{H} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \tag{2}$$

To compute $\mathbf{H}$, we select four points on the road image that form a known rectangle in the real world:

- Far-left road point: $P_1 = (330, 450)$
- Far-right road point: $P_2 = (1240, 450)$
- Near-right road point: $P_3 = (1310, 750)$
- Near-left road point: $P_4 = (300, 750)$

These points cover two standard traffic lanes. In reality, standard highway lanes are approximately 3.5 meters wide each, giving a total width of 7.0 meters. The distance along the lane dash markings between row $y = 450$ and row $y = 750$ corresponds to 3.5 meters. With these four known image coordinates and their matching real-world meter coordinates, OpenCV calculates the transform matrix using `cv2.getPerspectiveTransform`.

D. Ground-Contact Point and Velocity Smoothing

Cars have height, so using the center of the bounding box can cause errors as the car tilts or changes perspective. Instead, we pick the bottom-center point of the bounding box:

$$u_{\text{ground}} = \frac{x_1 + x_2}{2}, \quad v_{\text{ground}} = y_2 \tag{3}$$

This point corresponds to where the rear tires touch the road pavement. We pass this point through our homography matrix to get the vehicle position (X_t, Y_t) in meters.

To calculate speed, we save the ground positions of each vehicle over a rolling window of 10 frames ($\Delta t \approx 0.33$ seconds). The distance moved between the oldest and newest sample in the window is:

$$\text{Distance } (d) = \sqrt{(X_{\text{new}} - X_{\text{old}})^2 + (Y_{\text{new}} - Y_{\text{old}})^2} \tag{4}$$

The raw speed in km/h is then calculated as:

$$\text{Speed}_{\text{raw}} = \left(\frac{d}{\Delta t} \right) \times 3.6 \tag{5}$$

Because small bounding box jitters can create brief speed spikes, we apply an exponential moving average (EMA) filter:

$$\text{Speed}_{\text{smooth}} = 0.3 \times \text{Speed}_{\text{raw}} + 0.7 \times \text{Speed}_{\text{prev}} \tag{6}$$

Any speed reading below 5 km/h or above 180 km/h is treated as an edge artifact and discarded.

E. Virtual Tripwire Counting and CSV Export

To count traffic volume without double counting, we place a virtual counting line across the road at vertical position $y = 594$ pixels. When the bottom point of a vehicle crosses this line, its track ID is recorded in a counted set, and the vehicle count for that class is increased.

During the video run, the system exports four clean CSV files:

- `tracked_detections.csv`: Stores bounding boxes and timestamps for every frame.
- `vehicle_speed_readings.csv`: Stores rolling speed readings for each vehicle.
- `vehicle_speed_summary.csv`: Stores each vehicle average speed, maximum speed, and whether it went over the speed limit.
- `traffic_counts_timeseries.csv`: Groups vehicle counts into 5-second buckets for timeline charts.

IV. Experimental Results and Discussion

A. Experimental Setup

We tested our system on a 1080p highway video recorded at 30 frames per second on a Windows laptop with an Intel processor and 16 GB RAM. The speed limit was set to 60.0 km/h. Table I summarizes the parameters used.

TABLE I
System Parameter Settings

Parameter	Value
Video Resolution	1920×1080 pixels
Frame Rate	30 FPS
Object Detector	YOLOv8n (COCO weights)
Confidence Threshold	0.30
Speed Smoothing Window	10 frames
Smoothing Factor (α)	0.30
Speed Limit Threshold	60.0 km/h
Counting Line Position	$y = 594$ px
Lane Width $\times$ Length	$7.0\,\text{m} \times 3.5\,\text{m}$

B. Tracking and Speed Performance

When we ran the earlier flat pixel method without homography, the system reported impossible speeds above 250 km/h because of the camera perspective. After applying our homography calibration, the speeds matched realistic highway values.

The system processed 659 frames of the video and produced the following traffic summary:

- **Total unique vehicles counted by tripwire:** 29 vehicles (26 cars and 3 motorcycles).
- **Total tracks detected across all frames:** 37 vehicle tracks.
- **Average fleet speed:** 41.24 km/h.
- **Maximum recorded speed:** 71.10 km/h.
- **Over-speeding vehicles (> 60 km/h):** 5 vehicles.

Table II breaks down the speed and count results across vehicle types.

TABLE II
Vehicle Count and Speed Performance Summary

Vehicle Class	Count	Average Speed (km/h)	Over-Limit (> 60 km/h)
Car	26	43.02	4
Motorcycle	3	45.38	1
Bus	0 (tripwire)	30.73 (partial)	0
Truck	0	–	0
Total / Fleet	**29 / 37**	**41.24**	**5**

Cars traveled at an average speed of 43.02 km/h, while motorcycles were slightly faster with an average speed of 45.38 km/h. A bus was detected briefly at the top of the frame moving at 30.73 km/h before exiting, but it did not cross the downstream counting line.

C. Tableau Visualization Analysis

We built an interactive dashboard in Tableau Desktop using the exported CSV files, as shown in Fig. 2. The dashboard is divided into key analytical cards:

1) **Top KPI Cards:** Clearly display the total vehicle tracks (37), the average speed (41.24 km/h), the maximum speed (71.10 km/h), and total over-limit violations (5).
2) **Traffic Volume Over Time:** A line chart displaying vehicle flow across 5-second time buckets. It shows traffic peaks around the 5-second mark and gradually levels off.
3) **Vehicle Count by Type:** A bar chart showing the composition of traffic, highlighting that cars form the majority of road users (26 counted).
4) **Average Speed by Type:** Compares vehicle categories, showing that motorcycles and cars travel at comparable normal driving speeds.
5) **Over-Limit Speed Readings:** A scatter plot of individual speed readings exceeding 60 km/h across the timeline. It clearly shows a motorcycle violation near the beginning ($t \approx 0.2$ s at 68.7 km/h) and a cluster of car speed readings between 60 km/h and 71.1 km/h around $t = 20$ seconds.
6) **Over-Limit Vehicles by Type:** A simple distribution chart indicating that four cars and one motorcycle violated the speed limit.

This dashboard gives traffic police and road authorities an easy way to understand traffic flow without having to watch hours of raw video recordings.

V. Conclusion and Future Work

In this project, we designed and built a working traffic monitoring system that combines YOLOv8n object detection, ByteTrack multi-object tracking, planar homography calibration, and Tableau visualization. By converting oblique camera pixels into real-world road meters, our system solved the severe perspective error of basic pixel methods and provided realistic vehicle speeds. The virtual counting line prevented

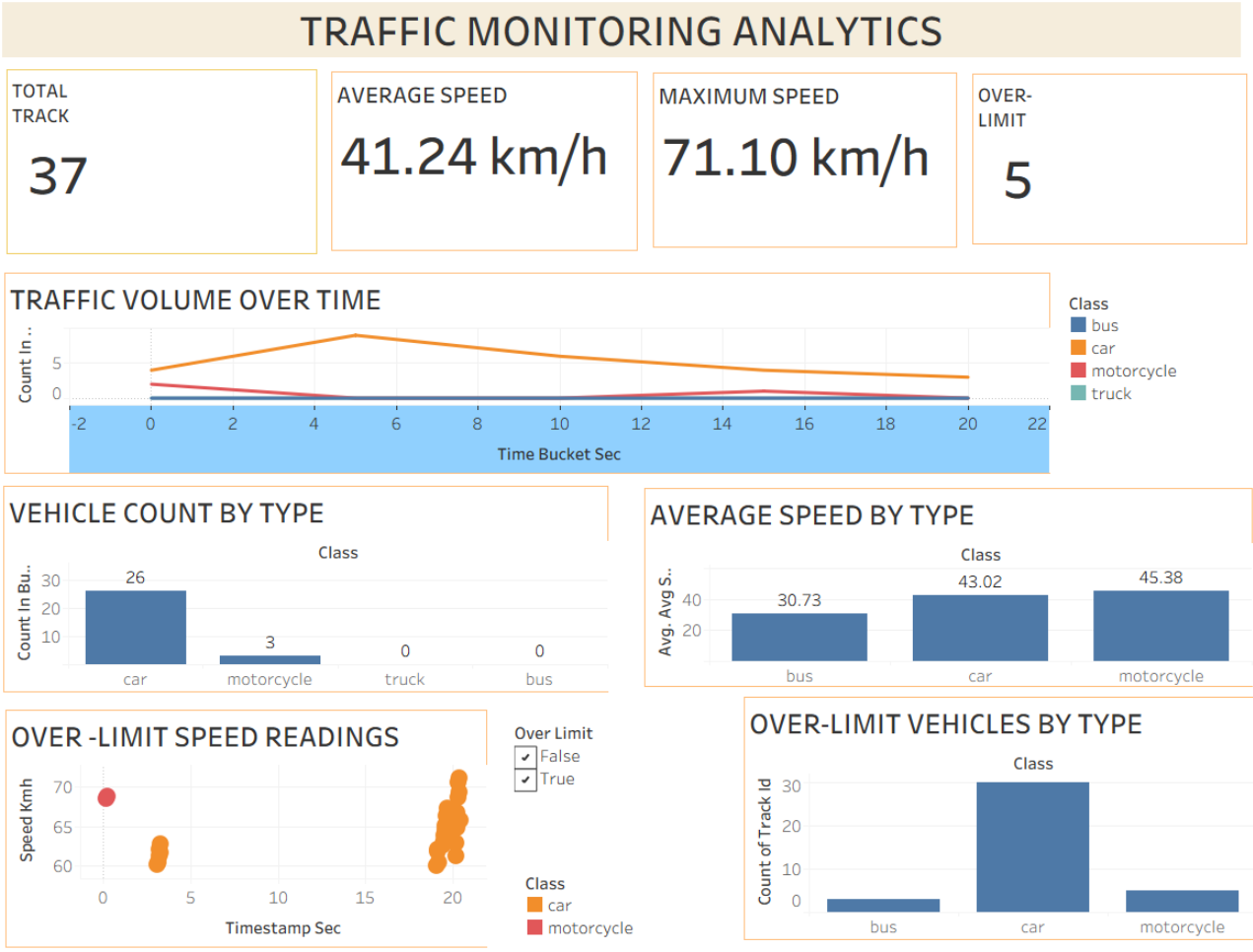


Fig. 2. Tableau Traffic Monitoring Analytics Dashboard showing summary KPIs, volume trends over time, count by vehicle class, average speed by vehicle type, over-limit speed readings, and total over-limit vehicles by class.

duplicate counting, and the exported data allowed us to create a clean, informative Tableau dashboard.

In future work, we plan to add automatic vanishing point detection from lane lines so that calibration can happen automatically without manual point selection. We also plan to test the system on nighttime and rainy video footage using contrast-enhancing filters.

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[2] Y. Zhang, P. Sun, Y. Dong, Z. Jiang, E. Pekhimenko, Q. Li, S. Shen, P. Luo, and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 1–21.

[3] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, 2016, pp. 3464–3468.

[4] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge Univ. Press, 2003.

[5] Z. He, Y. Chen, and B. Yao, "A vehicle speed estimation method based on monocular video and planar homography," *IEEE Access*, vol. 7, pp. 142 350–142 361, 2019.

[6] J. Heer, S. K. Card, and J. A. Landay, "Prefuse: A toolkit for interactive information visualization," in *Proc. SIGCHI Conf. Hum. Factors Comput. Syst. (CHI)*, 2008, pp. 421–430.